SYSTEM DESIGN DOCUMENTATION

DESIGN AND IMPLEMENTATION OF AN AI-ENHANCED WEB PORTAL. FOR A SWIMMING FACILITY WITH PRODUCTION DEPLOYMENT

Year 2026 **Vladimír Gavura**

TABLE OF CONTENTS

INTRODUCTION **7**

PURPOSE AND SCOPE OF THE DOCUMENT **8**

DEFINITIONS, ACRONYMS AND ABBREVIATIONS **9**

1. SUITABILITY ANALYSIS **12**

1.1. SWOT ANALYSIS 12

1.1.1. Strengths – Internal Factors *12*

1.1.2. Weaknesses – Internal Factors *12*

1.1.3. Opportunities – External Factors *13*

1.1.4. Threats – External Factors *13*

1.2. SMART ANALYSIS 13

1.2.1. Specific Objectives *13*

1.2.2. Measurable Objectives *14*

1.2.3. Achievable Objectives *14*

1.2.4. Relevant Objectives *15*

1.2.5. Time-Bound Objectives *15*

1.3. PEST ANALYSIS 15

1.3.1. Political and Legal Factors *15*

1.3.2. Economic Factors *16*

1.3.3. Social Factors *16*

1.3.4. Technological Factors *17*

1.4. SUITABILITY STUDY CONCLUSION 17

1.4.1. Feasibility *17*

1.4.2. Risks *18*

1.4.3. Market and Strategic Positioning *18*

1.4.4. Overall Evaluation *18*

2. REQUIREMENTS SPECIFICATION **19**

2.1. SYSTEM OVERVIEW 19

2.1.1. System Perspective *19*

2.1.2. Overview of System Functions *19*

2.1.3. User Characteristic *20*

2.1.4. System Constraints *20*

2.1.5. Assumptions and Dependencies *20*

2.2. FUNCTIONAL REQUIREMENTS 21

2.3. NON-FUNCTIONAL REQUIREMENTS 21
2.4. REQUIREMENTS CONFLICT ANALYSIS 23
2.5. VALIDATION CRITERIA 24

3. USE CASES 26

3.1. ACTOR IDENTIFICATION 26
3.2. USE CASE LIST 26
3.2.1. UC-01 – Display of Attendance Prediction 27
3.2.2. UC-02 – Assignment Course Preferences 28
3.2.3. UC-03 – Display Recommended Course 29
3.2.4. UC-04 – Manage Course Data 30
3.2.5. UC-05 – Configure System Parameters 31
3.2.6. UC-06 – Update Prediction 32
3.2.7. UC-07 – Display Model Evaluation 33
3.2.8. Use Case Diagram 34
3.2.9. Sequence diagrams 35
3.2.10. State Diagram – Prediction Update Process UC-06 37
3.2.11. Class Diagram of AI Component Architecture 39

4. USER INTERFACE DESIGN 40

5. SYSTEM ARCHITECTURE (THREE-TIER ARCHITECTURE) 42

5.1. PRESENTATION LAYER 42
5.1.1. Conceptual Design 42
5.1.2. Technical Implementation 43
5.2. APPLICATION LAYER 43
5.2.1. Conceptual Design 43
5.2.2. Technical Implementation 44
5.2.3. AI Components 44
5.3. DATA LAYER 45
5.3.1. Conceptual Design 45
5.3.2. Technical Implementation 45
5.4. DATA FLOW 46

6. SYSTEM DEPLOYMENT AND OPERATIONS 48

7. TEST CASE DESIGN FOR USE CASES 49

7.1. TESTING UC-01 – DISPLAY TRAFFIC PREDICTIONS 49
7.2. TESTING UC-02 – ASSIGNMENT COURSE PREFERENCES 50
7.3. TESTING UC-03 – DISPLAY RECOMMENDED COURSE 51

7.4. TESTING UC-04 – MANAGE COURSE 52
7.5. TESTING UC-05 – CONFIGURATE SYSTEM PARAMETERS 52
7.6. TESTING UC-06 – UPDATE PREDICTION 53
7.7. TESTING UC-07 – DISPLAY MODEL EVALUATION 54

CONCLUSION.....55

LIST OF TABLES AND FIGURES

Table 1 – Functional Requirements 21
Table 2 – Non-Functional Requirements - Performance 21
Table 3 - Non-Functional Requirements - Reliability 22
Table 4 - Non-Functional Requirements – Security and Data Protection 22
Table 5 - Non-Functional Requirements – Maintainability and Logging 22
Table 6 - Non-Functional Requirements – Data Backup 22
Table 7 - Non-Functional Requirements - Usability 23
Table 8 - Non-Functional Requirements – Model Accuracy 23
Table 9 – Requirements Conflict Analysis 23
Table 10 – Validation Criteria 24
Table 11 – Use Case Specification – Display of Attendance Prediction (UC-01) 27
Table 12 - Use Case Specification – Assignment Course Preferences (UC-02) 28
Table 13 - Use Case Specification – Display Recommended Course (UC-03) 29
Table 14 - Use Case Specification – Manage Course Data (UC-04) 30
Table 15 - Use Case Specification – Configure System Parameters (UC-05) 31
Table 16 - Use Case Specification – Update Prediction (UC-06) 32
Table 17 - Use Case Specification –Display Model Evaluation (UC-07) 33
Table 18 – Test Case – Prediction Endpoint Output and Performance Validation 49
Table 19 – Test Case – System Behavior During Prediction Service Unavailability 49
Table 20 – Test Case – Fallback to Last Available Prediction 50
Table 21 - Test Case – Submission of Valid Course Preferences 50
Table 22 – Test Case – Validation of Invalid Input Data 50
Table 23 – Test Case – Structure and Count of Recommended Course 51
Table 24 – Test Case – Behavior When No Suitable Course Are Available 51
Table 25 – Test Case – Create or Update Course 52
Table 26 – Test Case – Validation of Invalid Course Data 52
Table 27 – Test Case – Update Systems Configuration Parameter 52
Table 28 – Test Case – Validation of Invalid Parameter Value 53
Table 29 – Test Case – Successful Scheduled Prediction Update 53
Table 30 – Test Case – Behavior on Prediction Update Failure 53
Table 31 – Test Case – Display of Model Evaluation Metrics 54
Table 32 – Test Case – Behavior When Metrics Are Unavailable 54

Figure 1 – Use case diagram 35
Figure 2 - Sequence diagram UC-01 36
Figure 3 - Sequence diagram UC-02 a UC-03 37

FIGURE 4 - STATE DIAGRAM UC-06 38
FIGURE 5 - CLASS DIAGRAM 39
FIGURE 6 - HOMEPAGE DESIGN 40
FIGURE 7 - TRAFFIC PREDICTION MODULE DESIGN 40
FIGURE 8 – COURSE RECOMMENDATION MODULE DESIGN 41
FIGURE 9 - COURSE EVALUATION MODULE DESIGN 41
FIGURE 10- ENTITY-RELATIONSHIP DIAGRAM 46
FIGURE 11– THREE-TIER SYSTEM ARCHITECTURE 47

INTRODUCTION

Modern web applications increasingly integrate machine learning techniques in order to provide users with personalized services and decision support. In the domain of sports and recreational facilities, visitor traffic prediction and service recommendation represent a significant benefit in terms of both operational optimization and user experience.

The system is designed for a specific swimming facility, while all names and identifying details have been anonymized to protect sensitive information.

The objective of the system is to design and implement an extension of the swimming facility web portal with three core AI components:

- visitor traffic prediction on an hourly basis,
- course recommendation based on user preferences,
- detection of significant deviations between expected and actual attendance.

The system is designed as a standalone application module providing a REST API, which is integrated into the existing web environment. The solution follows the principles of a three-tier architecture and enforces a clear separation of concerns between the presentation, application, and data layers.

PURPOSE AND SCOPE OF THE DOCUMENT

The purpose of this system documentation is to formally and clearly specify the design, architecture, requirements, use cases, and testing approach of the system “AI-Enhanced Web Portal for a Swimming Facility”.

The document serves as:

- a foundation for system implementation,
- a reference document for stakeholders,
- a basis for system architecture and database design,
- a guideline for test case design.

The scope of the document includes:

- feasibility analysis (SWOT, SMART, PEST),
- specification of functional and non-functional requirements,
- analysis of requirement conflicts,
- definition of validation criteria,
- use case modeling,
- design of a three-tier system architecture,
- design of AI components (prediction, recommendation, anomaly detection),
- database design,
- system deployment in a cloud environment,
- test case design.

The document does not include detailed source code implementation.

DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Accuracy classification metric expressing the proportion of correctly classified instances.

Aggregation process of grouping data into higher-level summaries (e.g., aggregating individual records into hourly counts).

AI Artificial Intelligence – field of computer science focused on creating systems capable of performing tasks that require intelligent decision-making, such as prediction, classification, or anomaly detection.

Anomaly deviation from normal system behavior, e.g., significantly higher or lower values than expected.

API Application Programming Interface – interface that enables communication between software components through defined endpoints.

AWS Amazon Web Services – cloud platform providing infrastructure services such as computing power, storage, databases, and monitoring.

Backend server-side part of the system implementing application logic and AI components.

CI/CD Continuous Integration / Continuous Deployment – process of automating build, testing, and deployment of applications.

CloudWatch AWS service used for collecting, storing, and analyzing logs and system metrics.

Confusion Matrix evaluation matrix used for detailed analysis of classification models, showing correct and incorrect predictions.

Controller component of the application layer responsible for handling HTTP requests and delegating them to the service layer.

CRUD basic data operations: Create, Read, Update, Delete.

DB Database – persistent data storage of the system, implemented using a relational database (PostgreSQL).

Deployment process of deploying an application to a production or testing environment.

Docker tool enabling application containerization for consistent deployment and environment reproducibility.

Docker Container isolated runtime environment containing an application and its dependencies.

EC2 Elastic Compute Cloud – AWS service providing virtual servers for running applications.

ECS Elastic Container Service – AWS service for container orchestration and management.

Endpoint specific URL of a REST API providing a particular functionality.

Environment Variables configuration variables used to store sensitive data (e.g., credentials) securely outside the source code.

Fallback mechanism where the system returns the last available valid result in case of failure instead of an error.

FastAPI Python web framework for building high-performance REST APIs.

Feature input variable representing a single attribute influencing model prediction.

Frontend user-facing part of the system responsible for displaying outputs and enabling interaction.

GitHub Actions tool for automating CI/CD pipelines directly within a GitHub repository.

Grafana tool for visualizing metrics and building dashboards based on data from sources such as Prometheus.

HTTP HyperText Transfer Protocol – protocol used for data communication between client and server.

HTTPS secure version of HTTP using SSL/TLS encryption to protect transmitted data.

Horizontal Scaling approach to scaling by adding multiple instances of a service.

Isolation Forest anomaly detection algorithm identifying outliers based on data isolation.

JSON JavaScript Object Notation – data format used for exchanging structured data between frontend and backend.

Logging process of recording events, errors, and system behavior for diagnostics and monitoring.

MAE Mean Absolute Error – regression metric representing the average absolute difference between predicted and actual values.

Metrics quantitative measurements of system behavior (e.g., response time, number of requests, error rate).

ML Machine Learning – subset of artificial intelligence that uses algorithms to learn from data without explicit programming.

Model Metrics set of metrics used to evaluate model performance.

Monitoring process of tracking system health and performance using metrics.

ORM Object-Relational Mapping – technique for mapping application objects to relational database tables.

PostgreSQL open-source relational database management system used as the data layer.

Prediction Horizon time range for which the model generates predictions (in this project 7 days / 168 hours).

Prometheus open-source monitoring system used for collecting and storing time-series metrics.

RandomForestClassifier classification model based on an ensemble of decision trees.

RandomForestRegressor regression model based on an ensemble of decision trees.

RDS Relational Database Service – managed AWS service for running relational databases.

Repository component responsible for communication between application layer and database.

REST architectural style for designing web services using stateless communication over HTTP with standard methods (GET, POST, PUT, DELETE).

RMSE Root Mean Squared Error – regression metric sensitive to large errors, computed as the square root of the average squared differences.

Scalability ability of a system to handle increased load by adding resources.

Scheduler mechanism for executing tasks at predefined time intervals.

Service layer containing business logic and interaction with AI models.

Time Series chronologically ordered sequence of data points representing changes over time.

TimeSeriesSplit cross-validation method for time series data that preserves temporal order.

Timestamp time value representing a specific moment of data creation or update.

Train/Test Split method of dividing data into training and testing sets for objective model evaluation.

UI User Interface – graphical interface enabling user interaction with the system.

UX User Experience – overall user perception of system usability, clarity, and responsiveness.

Validation process of verifying correctness of inputs or model performance using test data.

Versioning process of assigning versions to models or APIs to track changes and enable rollback.

1. SUITABILITY ANALYSIS

1.1. SWOT Analysis

1.1.1. Strengths – Internal Factors

S1 – Iterative Model Improvement Capability

The system supports continuous improvement of AI models through iterative training and evaluation based on real operational data.

S2 – Clearly Defined Scope (Single Facility Deployment)

Focusing on a single swimming facility enables precise modeling, simplified integration, and reduced system complexity.

S3 – Control Over Data Model Design

The system processes real operational data provided by the facility, which are anonymized and aggregated into time intervals (e.g., hourly, seasonal patterns), ensuring both usability and compliance with data protection requirements.

S4 – Feedback Loop for Model Optimization

Deviations between predicted and actual attendance are stored and can be used to improve model performance over time.

S5 – Direct Integration into Web Environment

The AI components are directly integrated into the existing web portal via REST API, enabling practical real-world usage.

1.1.2. Weaknesses – Internal Factors

W1 – Limited Development Resources

The development is performed within a limited resource environment, which may impact development speed and testing depth.

W2 – Limited Historical Data in Early Stages

At initial deployment, the available historical dataset may be limited, which can affect model accuracy.

W3 – Narrow Functional Scope

The system focuses primarily on prediction and recommendation features without covering full operational functionalities (e.g., reservations, payments).

1.1.3. Opportunities – External Factors

O1 – Increasing Demand for Digital Services

Users increasingly expect real-time information about service availability and occupancy.

O2 – Accessibility of AI and Cloud Technologies

Modern AI frameworks and cloud platforms (e.g., AWS) significantly reduce the barrier to implementing intelligent systems.

O3 – Potential for Real Deployment

Collaboration with a real swimming facility enables deployment in a production environment.

O4 – Transferability to Other Domains

The solution can be adapted to similar domains such as fitness centers, sports facilities, or public venues.

1.1.4. Threats – External Factors

T1 – Existing Commercial Solutions

There are existing commercial systems that may offer more advanced or fully integrated solutions.

T2 – Regulatory Constraints (GDPR, AI Act)

Legal requirements related to data protection and AI transparency may impact system design and deployment.

T3 – Integration Limitations of Existing Infrastructure

The technical capabilities of the current web system may limit integration options.

T4 – User Trust in AI Predictions

Low prediction accuracy may reduce user trust and system adoption.

1.2. SMART Analysis

1.2.1. Specific Objectives

The system provides two primary AI-driven functionalities:

1. **Visitor Traffic Prediction**
   - generation of attendance predictions on an hourly basis for a minimum horizon of 7 days,

- input data consist of historical attendance records obtained from the operational system, anonymized and aggregated into time intervals,
- output is a time series of predicted values exposed via REST API and displayed within the web interface.

2. **Course Recommendation**
   - users (without registration) provide input parameters such as age, swimming skill level, and time availability,
   - the system returns the top-N most suitable courses (e.g., top-3) based on these preferences.
3. **Additionally, the system includes:**
   - detection of significant deviations between predicted and actual attendance,
   - generation of alerts when a defined threshold is exceeded.

1.2.2. Measurable Objectives

- Prediction performance is evaluated using standard regression metrics (MAE, RMSE) on a test dataset.
- Prediction results are publicly accessible via the web interface.
- Prediction updates are executed at a defined interval (e.g., once per day).
- The recommendation module returns a fixed number of results (e.g., top-3 courses).
- Alerts are generated when deviation exceeds a predefined threshold (e.g., difference ≥ defined value) and are stored in the database.

1.2.3. Achievable Objectives

- The system scope is limited to a single swimming facility, reducing implementation complexity.
- The system operates on aggregated attendance data, eliminating the need for complex data preprocessing.
- Standard machine learning techniques for time series prediction and recommendation are utilized.

- The development timeline allows for iterative implementation, testing, and model refinement.

1.2.4. Relevant Objectives

- The system does not require hardware development or integration with physical devices.
- The system does not process personal identifiable information (PII).
- The scope excludes non-core features such as payments, reservations, or mobile applications.
- The focus is strictly on AI functionality and its integration into a web-based environment.

1.2.5. Time-Bound Objectives

The system is planned to be implemented within an approximate timeframe of 5 months, divided into the following phases:

- requirements analysis and specification,
- system architecture and data model design,
- implementation of AI components,
- integration of the API into the web application,
- testing and evaluation of model performance,
- deployment, monitoring, and validation in a pilot production environment.

1.3. PEST Analysis

1.3.1. Political and Legal Factors

P1 – AI Regulation in the EU (AI Act)

The European Union is progressively introducing regulations governing artificial intelligence systems. The proposed solution is classified as a low-risk system, as it operates on aggregated, non-personal data. However, transparency must be ensured, including clear communication to users that predictions are generated by an AI model.

P2 – GDPR and Data Protection Compliance

The system is designed to process only aggregated attendance data (e.g., counts of entries and exits), minimizing GDPR-related risks. If future extensions include user-level data (e.g., course registrations), additional compliance measures will be required.

P3 – Digitalization Policies (EU and National Level)

Ongoing support for digital transformation in public and municipal services creates a favorable environment for deploying data-driven solutions such as this system.

1.3.2. Economic Factors

E1 – Budget Constraints of Public Facilities

Swimming facilities typically operate under limited budgets. Therefore, the system must be cost-efficient, avoiding expensive infrastructure and leveraging lightweight cloud solutions.

E2 – Cost of Cloud Infrastructure

Operational costs (hosting, database, monitoring) must remain sustainable. The system is designed to run on minimal infrastructure (e.g., single-instance deployment with optional scaling).

E3 – Variability in Demand

Economic conditions may influence visitor attendance patterns. The system must be able to adapt to changes in demand and evolving usage trends.

1.3.3. Social Factors

S1 – Increasing Demand for Self-Service Solutions

Users expect real-time access to information without requiring direct interaction with staff (e.g., phone calls or in-person inquiries).

S2 – Public Perception of AI Systems

Some users may be skeptical of AI-generated predictions. Therefore, the system should clearly communicate that outputs are estimates and not guaranteed values.

S3 – Changing Lifestyle and Sports Habits

Seasonal trends and lifestyle changes (e.g., increased interest in fitness and wellness) significantly affect attendance patterns.

S4 – Demographic Structure of the Region

Age distribution and population characteristics influence demand for specific courses and services.

1.3.4. Technological Factors

T1 – Availability of Machine Learning Tools

There is wide availability of mature open-source tools for time series forecasting and recommendation systems, enabling efficient development.

T2 – Web Accessibility and Internet Usage

High internet penetration supports widespread adoption of web-based solutions.

T3 – Integration with Existing Web Infrastructure

The level of maturity of the existing web system may impact integration complexity. The use of REST API minimizes coupling and simplifies integration.

T4 – Rapid Evolution of AI Technologies

AI technologies evolve rapidly, providing opportunities for model improvement, but also requiring consideration of long-term maintainability.

T5 – Cloud and DevOps Ecosystem

Modern cloud platforms (e.g., AWS) and DevOps tools (CI/CD, containerization, monitoring) enable efficient deployment, scalability, and system observability.

1.4. Suitability Study Conclusion

The feasibility study, based on the SWOT, SMART, and PEST analyses, confirms that the proposed AI-enhanced extension of the swimming facility web portal is feasible, carries an acceptable level of risk, and is technologically justified.

1.4.1. Feasibility

The system is characterized by:

- a clearly defined scope (single swimming facility),
- a realistic implementation timeline,
- availability of real operational data,
- measurable outputs (MAE/RMSE, top-N recommendations, logged deviations).

The system does not require hardware development or complex infrastructure, which significantly reduces technical complexity.

1.4.2. Risks

The main identified risks include:

- quality and representativeness of input data,
- prediction accuracy under seasonal variations,
- potential integration limitations with existing web infrastructure,
- the need for transparency towards users (AI regulation, GDPR compliance).

1.4.3. Market and Strategic Positioning

Although commercial solutions for occupancy monitoring exist, the system is not intended to compete with full-scale enterprise systems. Instead, it focuses on:

- implementation of AI models within a specific domain,
- practical deployment of AI components,
- integration with a real web-based environment.

From a technological perspective, the system aligns with current trends in digitalization and the adoption of AI in public services.

1.4.4. Overall Evaluation

The system is:

- technically feasible,
- manageable within the given timeframe,
- compliant with legal requirements,
- capable of delivering real practical value for a specific facility.

The feasibility study confirms that proceeding to the detailed requirements specification phase is justified.

2. REQUIREMENTS SPECIFICATION

This chapter defines the requirements of the system designed as an AI-enhanced extension of a swimming facility web portal. It describes system functionality, constraints, and characteristics without detailing implementation specifics.

The purpose of this specification is to clearly define what the system shall provide to end users and system operators, and to establish a foundation for system design, implementation, and testing.

The scope of the system includes:

- processing of swimming facility attendance data (inputs and outputs),
- generation of visitor traffic predictions,
- course recommendation based on user-defined preferences,
- detection of significant deviations between predicted and actual attendance,
- presentation of results through a web interface.

The document is intended for:

- system developers,
- project stakeholders,
- personnel responsible for testing and evaluation.

2.1. System Overview

2.1.1. System Perspective

The proposed system represents an extension of an existing swimming facility web application implemented as a standalone module exposing a REST API. The web application consumes this API to retrieve visitor traffic predictions and course recommendations, which are then presented to users.

This approach minimizes changes to the existing system and enables independent development, testing, and deployment of AI functionalities.

2.1.2. Overview of System Functions

The system provides the following core functionalities:

- processing and aggregation of attendance data,

- generation of visitor traffic predictions on an hourly basis for at least 7 days ahead,
- exposure of predictions via REST API,
- recommendation of courses based on user preferences (e.g., age, skill level, availability),
- detection of significant deviations between predicted and actual attendance,
- recording and reporting of detected anomalies for administrative purposes.

2.1.3. User Characteristics

The system distinguishes the following user roles:

Public User - a visitor of the web application who can view traffic predictions and receive course recommendations without registration.

Administrator - an authorized user responsible for managing course data, configuring system parameters, and monitoring system performance and model evaluation.

2.1.4. System Constraints

The system shall operate under the following constraints:

- it is designed for a single swimming facility and its operational conditions,
- prediction outputs represent estimates and may differ from actual values,
- the system does not include development or integration of hardware devices.

2.1.5. Assumptions and Dependencies

The system relies on the following assumptions:

- availability of attendance data in regular time intervals,
- availability of course data including attributes such as age group, skill level, and schedule,
- availability of a web application capable of consuming REST API responses,

- continuous availability of backend services responsible for data processing and prediction generation.

2.2. Functional Requirements

Table 1 – Functional Requirements

Functional Requirements	
ID	**The system shall provide the following functionalities:**
#FP01	The system shall generate visitor traffic predictions on an hourly basis for a minimum prediction horizon of 7 days (168 hours).
#FP02	The system shall process and aggregate attendance data into defined time intervals (e.g., hourly) for use in prediction models.
#FP03	The system shall recommend the three most suitable courses based on user input, including age, swimming skill level, and time availability.
#FP04	The system shall display recommended courses, including basic information.
#FP05	The system shall identify significant deviations between expected and actual attendance.
#FP06	The system shall notify staff about detected deviations.
#FP07	The system shall store generated alerts in the database.
#FP08	The system shall allow updating of course data.
#FP09	The system shall allow administrators to modify basic system parameters.
#FP10	The system shall provide an interface for retrieving visitor traffic predictions.
#FP11	The system shall provide an interface for retrieving recommended courses.
#FP12	The system shall process input and output data in defined time intervals.
#FP13	The system shall store processed data for further evaluation.

2.3. Non-Functional Requirements

Table 2 – Non-Functional Requirements - Performance

Performance		
ID	**Requirement**	**Measurability**

#NP01	The system shall provide an API response for prediction within 3 seconds under normal load.	Response time < 3 s
#NP02	Visitor traffic prediction shall be updated at least once per day.	Verification of update interval

Table 3 - Non-Functional Requirements - Reliability

Reliability		
ID	**Requirement**	**Measurability**
#NP03	The system shall ensure stable operation during normal usage without frequent failures.	Functional testing
#NP04	In case of a temporary failure, the system shall allow displaying the last available prediction.	Simulated failure test

Table 4 - Non-Functional Requirements – Security and Data Protection

Security and Data Protection		
ID	**Requirement**	**Measurability**
#NP05	Communication between the web application and the API shall be encrypted.	HTTPS
#NP06	The system shall not process or store personally identifiable data of visitors.	Data model verification
#NP07	Access to the administrative part of the system shall be protected by authentication.	Access control testing

Table 5 - Non-Functional Requirements – Maintainability and Logging

Maintainability and Logging		
ID	**Requirement**	**Measurability**
#NP08	The system shall log errors and significant events.	Log inspection
#NP09	The system shall allow model updates without requiring changes to the presentation layer.	Update testing

Table 6 - Non-Functional Requirements – Data Backup

Data Backup

ID	Requirement	Measurability
#NP10	The system shall enable database backups at regular intervals defined by system configuration.	Verification of backup mechanism

Table 7 - Non-Functional Requirements - Usability

Usability		
ID	**Requirement**	**Measurability**
#NP11	Visitor traffic prediction shall be presented in a clear and understandable way for a general user.	User testing
#NP12	The system shall clearly indicate that predictions are estimates and may differ from actual values.	UI validation

Table 8 - Non-Functional Requirements – Model Accuracy

Model Accuracy		
ID	**Requirement**	**Measurability**
#NP13	The prediction model shall achieve an acceptable level of accuracy defined by MAE or RMSE on a test dataset.	Experimental evaluation

2.4. Requirements Conflict Analysis

This chapter identifies potential conflicts between system requirements. A conflict represents a situation where fulfilling one requirement may negatively impact another. The objective is to explicitly identify these conflicts and define an appropriate compromise.

Table 9 – Requirements Conflict Analysis

Requirements Conflict Analysis			
ID	**Conflicting Requirements**	**Conflict Description**	**Proposed Solution / Compromise**
#K01	NP01 vs. NP13	Higher prediction accuracy may require more computationally intensive	Predictions are computed periodically (e.g., every 15

		processing, which can increase API response time.	minutes), and the API returns precomputed results.
#K02	NP02 vs. NP03	Frequent prediction updates increase the risk of errors and system instability.	Updates are performed at fixed intervals, and in case of failure, the last available prediction is retained (NP04).
#K03	NP06 vs. NP13	Minimizing processed data (without personal data) may limit the ability to improve model accuracy.	Only aggregated data (counts and time intervals) are stored and evaluated without personal identification.
#K04	NP11 vs. NP08	A simple user interface may conflict with the need to display technical error information.	Technical errors are logged (NP08), while users are shown only clear and understandable status information.

2.5. Validation Criteria

Validation criteria define how selected functional and non-functional requirements are verified.

Table 10 – Validation Criteria

Validation Criteria	
Requirement ID	**Verification Method**
FP01	API call for prediction and verification that the output contains a 7-day prediction (168 hours).
FP03	Input of test data (age, skill level, time) and verification that the system returns three recommended courses.
FP07	Simulation of deviation between prediction and actual values and verification that it is recorded in the system.

NP01	Repeated API calls and measurement of response time (must be < 3 s).
NP02	Verification of prediction update timestamp.
NP11	User testing to verify clarity of prediction display.
NP13	Splitting data into training and testing sets and calculating MAE or RMSE.

3. USE CASES

After defining the system objectives and specifying functional and non-functional requirements, use cases were identified to describe system behavior from the perspective of different actors.

Use cases serve as a foundation for system architecture design, data modeling, and the implementation of AI components.

- The defined use cases cover the main system functionalities:
- display of attendance prediction,
- course recommendation based on user preferences,
- system administration,
- model evaluation.

Each use case is described using a structured table including actors, preconditions, postconditions, main flow, and alternative scenarios.

3.1. Actor Identification

Public User - A visitor of the swimming facility website who can view attendance predictions and receive course recommendations without registration.

Administrator - An authorized user responsible for managing course data, configuring system parameters, and monitoring model performance.

System (Scheduled Task) - An automated process that periodically processes data and updates predictions without direct user interaction.

3.2. Use Case List

UC-01 – Display of Attendance Prediction

UC-02 – Assignment of Course Preferences

UC-03 – Display Recommended Course

UC-04 – Manage Course Data

UC-05 – Configure System Parameters

UC-06 – Update Prediction

UC-07 – Display Model Evaluation

3.2.1. UC-01 – Display of Attendance Prediction

Table 11 – Use Case Specification – Display of Attendance Prediction (UC-01)

Name	Display of Attendance Prediction
ID	UC-01
Actor	Public User
Description	The user views the current attendance prediction on an hourly basis for the next 7 days to choose an appropriate time for visiting the swimming facility.
Precondition	• The user opens the main webpage of the swimming facility • The system has the latest generated prediction stored in the database • The API service is available
Postcondition	• The prediction is displayed on the webpage (e.g., graph or table) • The user is informed that the prediction is an estimate • In case of failure, a user-friendly error message is displayed
Main Flow	1. The user opens the main webpage 2. The web application sends a request to the API 3. The API retrieves prediction data from the database 4. The API returns predicted values (hourly for 7 days) 5. The web application displays the prediction 6. The user can switch between days
Alternative Flows and Exception	**A1 – Prediction not available** • API returns an error or empty response • UI displays message: "Prediction is temporarily unavailable" • Error is logged **A2 – Last prediction available** • API returns last stored prediction • UI displays prediction with timestamp of last update

Priority	High
Non-Functional Requirements	• NP01 • NP05 • NP11 • NP12
Assumptions	• Prediction is generated periodically, • Historical data is available

3.2.2. UC-02 – Assignment of Course Preferences

Table 12 - Use Case Specification – Assignment Course Preferences (UC-02)

Name	Assignment of Course Preferences
ID	UC-02
Actors	Public User
Description	The user enters preferences (age, skill level, time availability) to receive course recommendations.
Preconditions	• User is on homepage or recommendation section • Course data is available • Recommendation API is available
Postconditions	• Preferences are processed • System returns Top 3 courses or error
Main Flow	1. User fills form: • age • skill level • time preference 2. User submits form 3. Web sends data to API 4. API evaluates courses 5. API returns recommendations 6. UI displays results
Alternative flows and Exceptions	**A1 – Invalid input**

	• Validation error • User is asked to correct form **A2 – No course found** • API returns empty result • UI shows message **A3 – API unavailable** • UI shows error message • Error logged
Priority	High
Non-Functional Requirements	• NP01 • NP05 • NP11
Assumption	• Course data exists • Model is available

3.2.3. UC-03 – Display Recommended Course

Table 13 - Use Case Specification – Display Recommended Course (UC-03)

Name	Display Recommended Courses
ID	UC-03
Actor	Public User
Description	The user receives a list of the most suitable courses based on preferences.
Precondition	• UC-02 completed • Course data available • Model available
Postcondition	• Up to 3 courses displayed • Basic course info shown • Error handled if needed
Main Flow	1. System receives API response 2. UI processes data 3. Courses are displayed

	4. User can view details
Alternative Flows and Exceptions	**A1 – Less than 3 courses** • Only available courses shown **A2 – No courses** • Message displayed **A3 – API error** • Error message + logging
Priority	High
Non-Functional Requirements	• NP01 • NP05 • NP11
Assumptions	• Data valid • Model initialized

3.2.4. UC-04 – Manage Course Data

Table 14 - Use Case Specification – Manage Course Data (UC-04)

Name	Manage Course Data
ID	UC-04
Actor	Administrator
Description	Administrator manages course data (create, update, delete).
Precondition	• Admin logged in • DB available
Postcondition	• Data stored • Available for model • Feedback shown
Main Flow	1. Admin opens course section 2. System shows courses 3. Admin selects action: • create • edit • delete

	4. Admin edits data 5. Confirms changes 6. System saves data 7. Confirmation shown
Alternative Flows and Exceptions	**A1 – Invalid data** • Validation error **A2 – DB error** • Error message + log
Priority	High
Non-Functional Requirements	• NP05 • NP07 • NP08
Assumptions	• Admin permissions exist • DB structure ready

3.2.5. UC-05 – Configure System Parameters

Table 15 - Use Case Specification – Configure System Parameters (UC-05)

Name	Configure System Parameters
ID	UC-05
Actor	Administrator
Description	Admin modifies system parameters affecting prediction and processing.
Precondition	• Admin logged in • System + DB available
Postcondition	• Parameters updated • Used in next processing
Main Flow	1. Admin opens settings 2. System displays parameters 3. Admin modifies values 4. Confirms changes 5. System validates values

	6. Saves changes 7. Shows confirmation
Alternative Flows and Exceptions	**A1 – Invalid value** • Rejected **A2 – Save error** • Error message + log
Priority	Medium
Non-Functional Requirements	• NP05 • NP07 • NP08
Assumptions	• Parameter ranges defined

3.2.6. UC-06 – Update Prediction

Table 16 - Use Case Specification – Update Prediction (UC-06)

Name	Update Prediction
ID	UC-06
Actor	System (Scheduled Task)
Description	The system periodically updates the prediction using historical data.
Precondition	• Historical data available • Scheduler configured • DB available
Postcondition	• New prediction generated • Stored in DB • Available via API
Main Flow	1. Scheduler starts process 2. Load historical data 3. Data preprocessing 4. Model update / reuse 5. Generate prediction 6. Store prediction

	7. Expose via API
Alternative Flows and Exceptions	**A1 – Data unavailable** • Old prediction kept **A2 – Model failure** • Old prediction kept **A3 – DB error** • Error logged
Priority	High
Non-Functional Requirements	• NP02 • NP04 • NP08 • NP10 • NP13
Assumptions	• Data quality sufficient • Scheduler configured

3.2.7. UC-07 – Display Model Evaluation

Table 17 - Use Case Specification –Display Model Evaluation (UC-07)

Name	Display Model Evaluation
ID	UC-07
Actor	Administrator
Description	Admin views model performance metrics (MAE, RMSE).
Preconditions	• Admin logged in • Model evaluated • Metrics stored
Postconditions	• Metrics displayed • Timestamp shown
Main Flow	1. Admin opens evaluation section 2. System loads metrics 3. Displays MAE/RMSE + timestamp 4. Admin evaluates performance

Alternative Flows and Exceptions	**A1 – No data** • Message displayed **A2 – DB error** • Error + log
Priority	Medium
Non-Functional Requirements	• NP05 • NP07 • NP08 • NP13
Assumptions	• Metrics computed

3.2.8. Use Case Diagram

Use case diagram illustrates the functional scope of the AI-enhanced system from the perspective of individual actors and their interactions with the system.

The diagram identifies three primary actors:

- Public User, who interacts with the system through the web interface,
- Administrator, responsible for system configuration and data management,
- System (Scheduled Task), representing automated background processes.

The Public User is associated with use cases related to viewing attendance predictions and receiving course recommendations based on provided preferences.

The Administrator performs management operations, including:

- course data management,
- configuration of system parameters,
- monitoring of model performance.

The System actor represents automated execution of periodic processes, specifically the update of attendance predictions (UC-06).

The diagram provides a high-level overview of system functionality and serves as a conceptual link between requirements specification and detailed behavioral models (sequence diagrams).

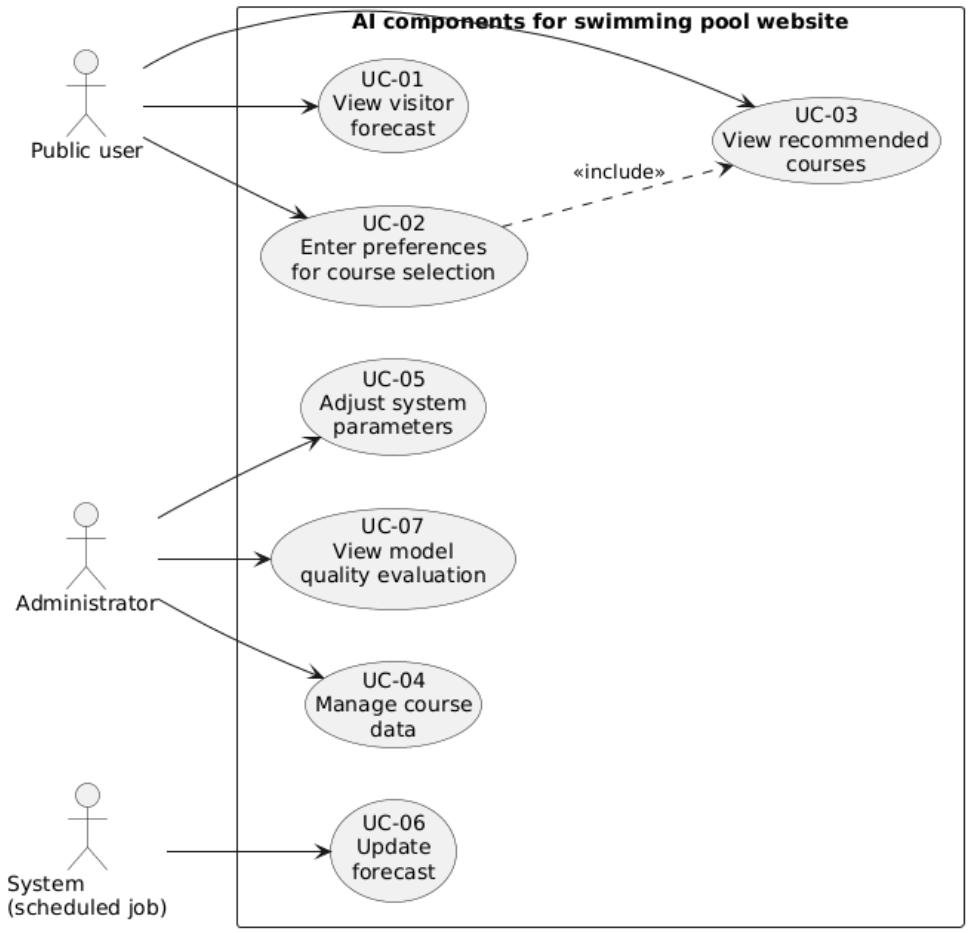

Figure 1 – Use case diagram

3.2.9. Sequence diagrams

Sequence diagrams represent the dynamic behavior of the system by illustrating the order of interactions between system components during the execution of selected use cases.

They describe:

- the flow of requests from the user interface to the backend,
- processing logic within the application layer,
- communication with the data layer,
- and the system response returned to the user.

These diagrams also capture alternative scenarios, including error handling and fallback mechanisms, which are essential for ensuring system reliability.

Sequence diagram UC-01 models the process of retrieving and displaying attendance prediction data. The interaction begins when the user accesses the web

interface, which sends a request to the REST API endpoint responsible for prediction retrieval. The API processes the request by accessing the database and retrieving the latest available prediction. The result is then returned to the presentation layer, where it is visualized (e.g., as a graph or table).

The diagram also includes alternative scenarios:

- If the prediction service is unavailable, the system returns an appropriate error response.
- If a previously generated prediction exists, the system uses a fallback mechanism and returns the last available prediction along with its timestamp.

This ensures that the system remains functional even in case of partial failures.

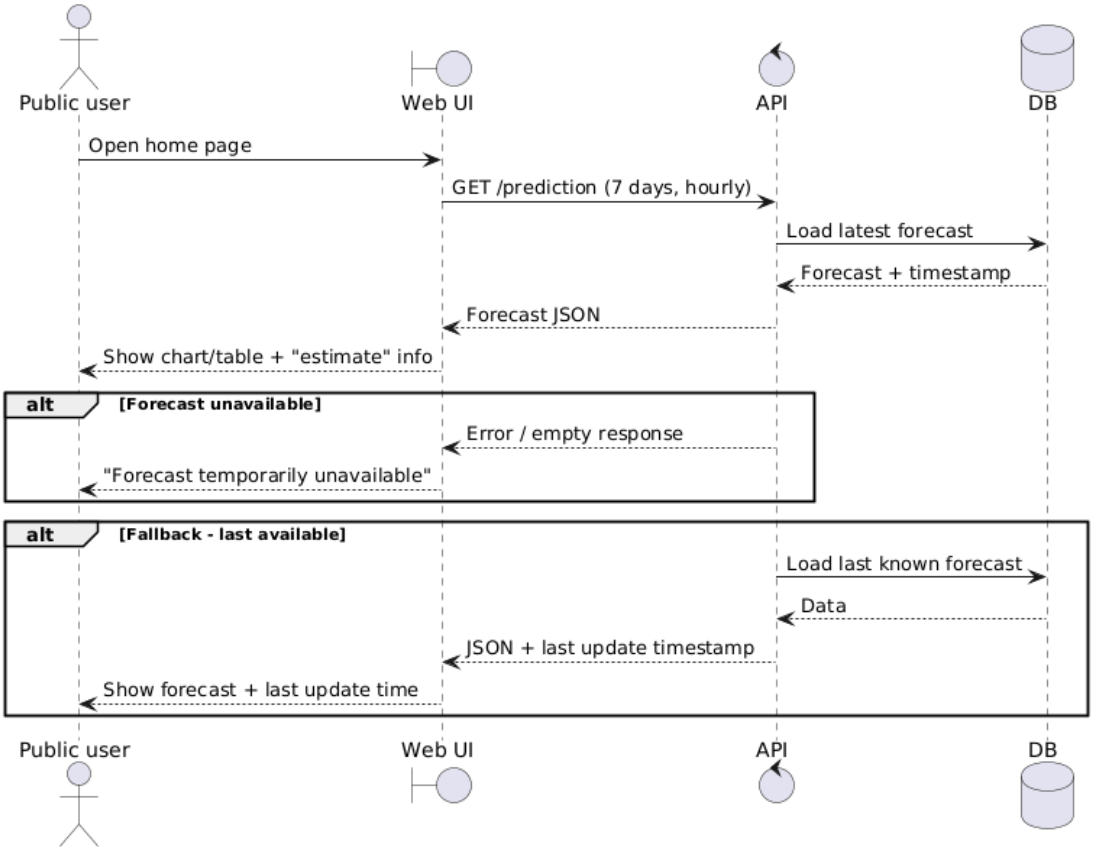

Figure 2 - Sequence diagram UC-01

Sequence diagram UC-02 a UC-03 represents the process of course recommendation based on user preferences. The interaction starts when the user submits input data (e.g., age, skill level, and time availability) through the web interface. These data are sent to the recommendation API. The backend system retrieves available course data from the database and processes the input using the recommendation module. The

model evaluates the suitability of individual courses and returns a ranked list of the most appropriate options. The result is then displayed to the user through the presentation layer.

The diagram also includes validation and edge-case scenarios:

- invalid input data are detected and rejected before processing,
- if no suitable course is found, the system returns an empty result with a corresponding message.

This ensures robustness and usability of the recommendation feature.

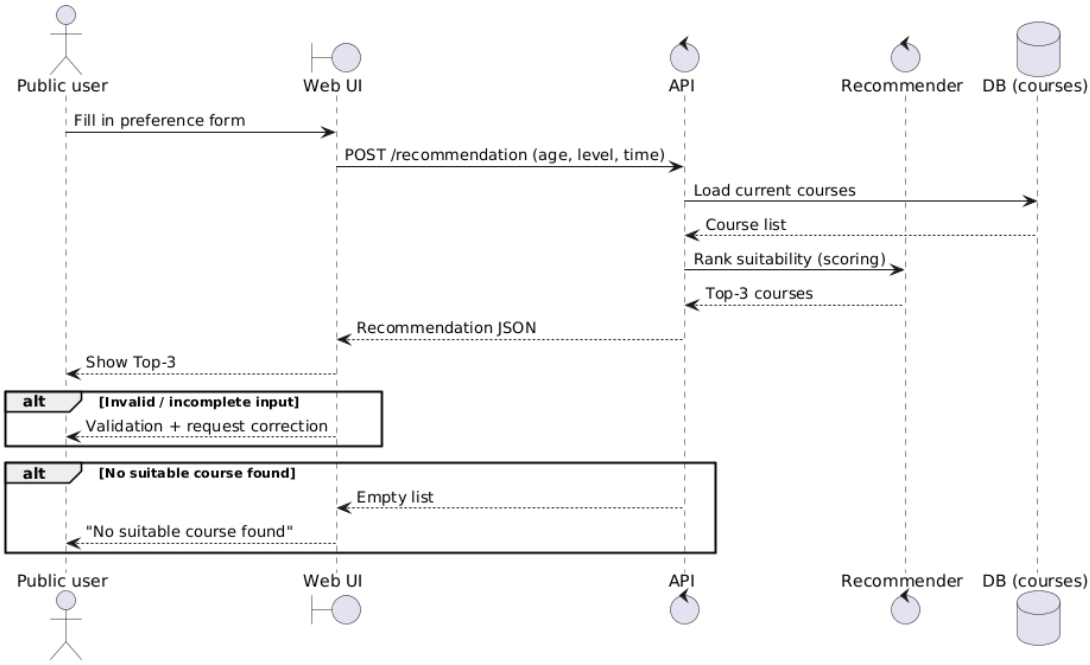

Figure 3 - Sequence diagram UC-02 and UC-03

3.2.10. State Diagram – Prediction Update Process UC-06

The state diagram illustrates the lifecycle of the prediction update process (UC-06), which is executed periodically by the system scheduler. The process begins with the initialization state, followed by data loading from the database. The system then proceeds to data preprocessing and model update or reuse. After successful processing, the system generates a new prediction and stores the result in the database. The prediction then becomes available for API requests.

The diagram also models error states:

- if data are unavailable or processing fails, the system transitions to an error state,
- if a previous prediction exists, a fallback mechanism is activated,

- otherwise, the prediction is marked as unavailable.

The process is cyclic and is repeated during the next scheduled execution.

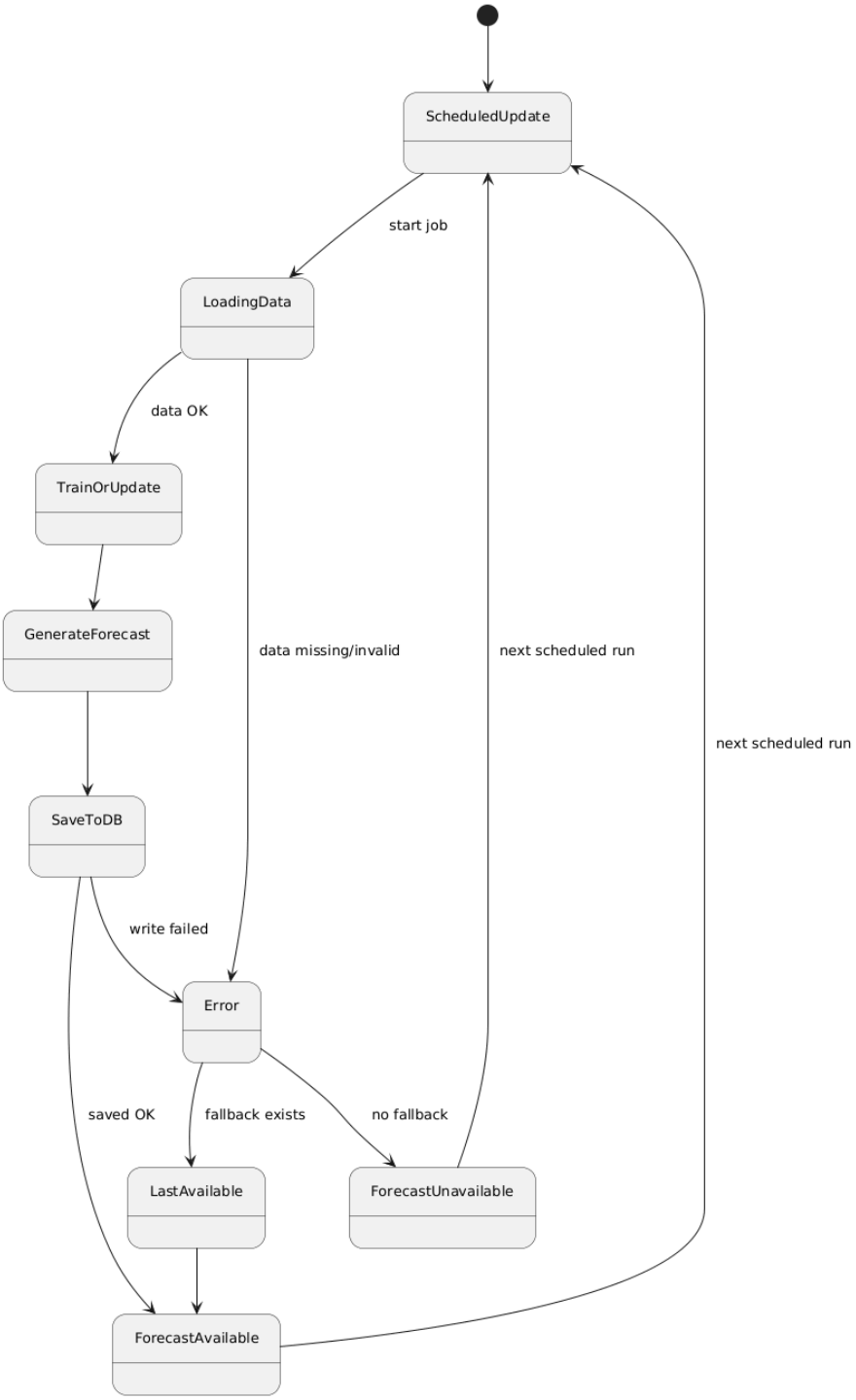

Figure 4 - State diagram UC-06

3.2.11. Class Diagram of AI Component Architecture

The class diagram represents the structural design of the AI component architecture and its integration within the application layer.

It illustrates the relationships between key system components, including:

- controllers responsible for handling API requests,
- service layer containing business logic and AI models,
- repository layer providing access to the database,
- domain entities representing system data.

The diagram reflects the layered architecture of the system (Controller–Service–Repository pattern) and demonstrates separation of concerns between different responsibilities.This structural design supports maintainability, scalability, and extensibility of the system.

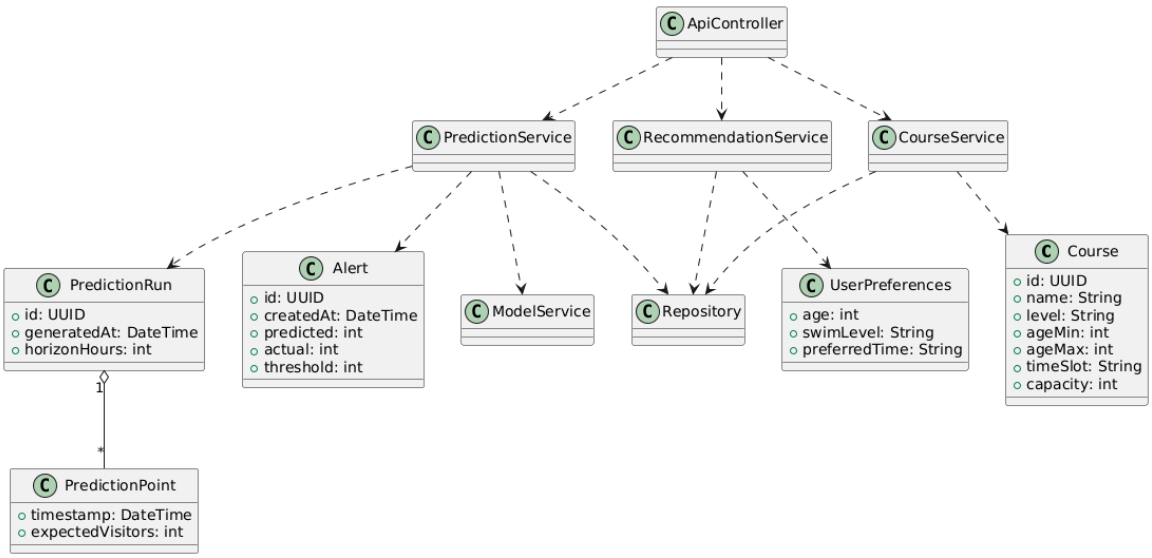

Figure 5 - Class diagram

4. USER INTERFACE DESIGN

The graphical design of the user interface represents the visual representation of the proposed web application and the way individual functional modules of the system are presented.

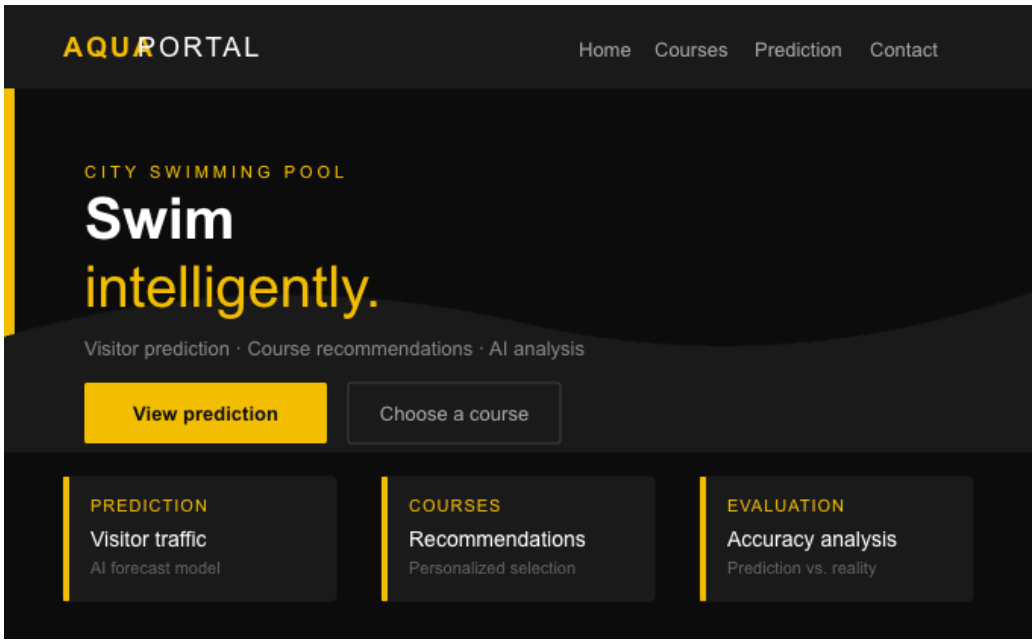

Figure 6 - Homepage design

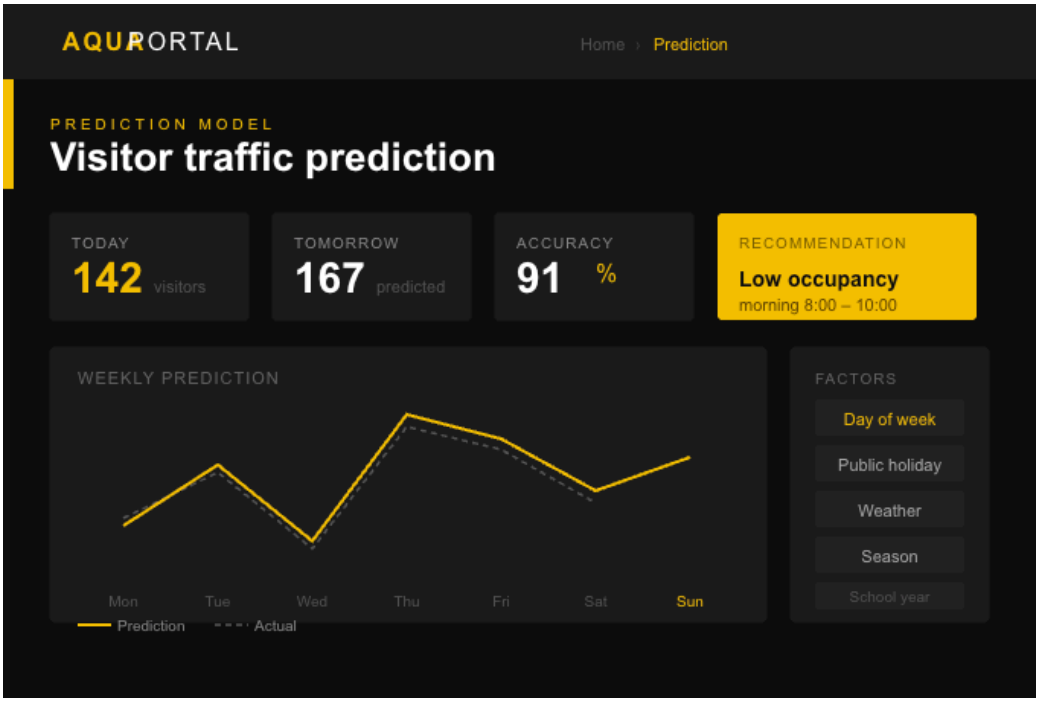

Figure 7 - Traffic prediction module design

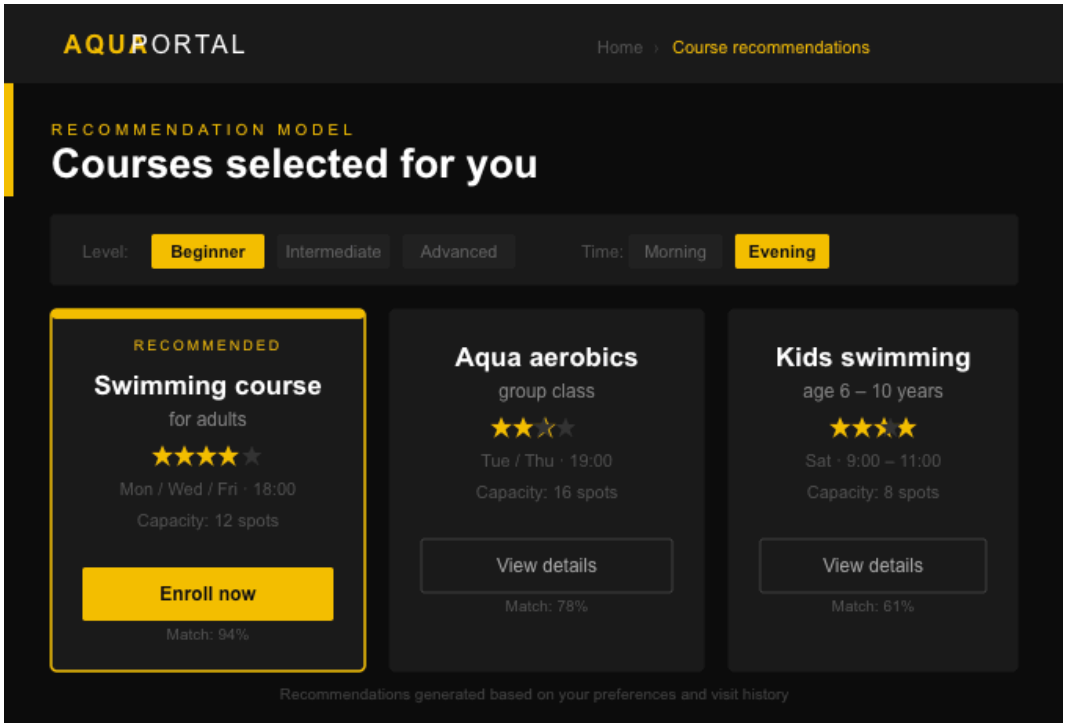

Figure 8 – Course recommendation module design

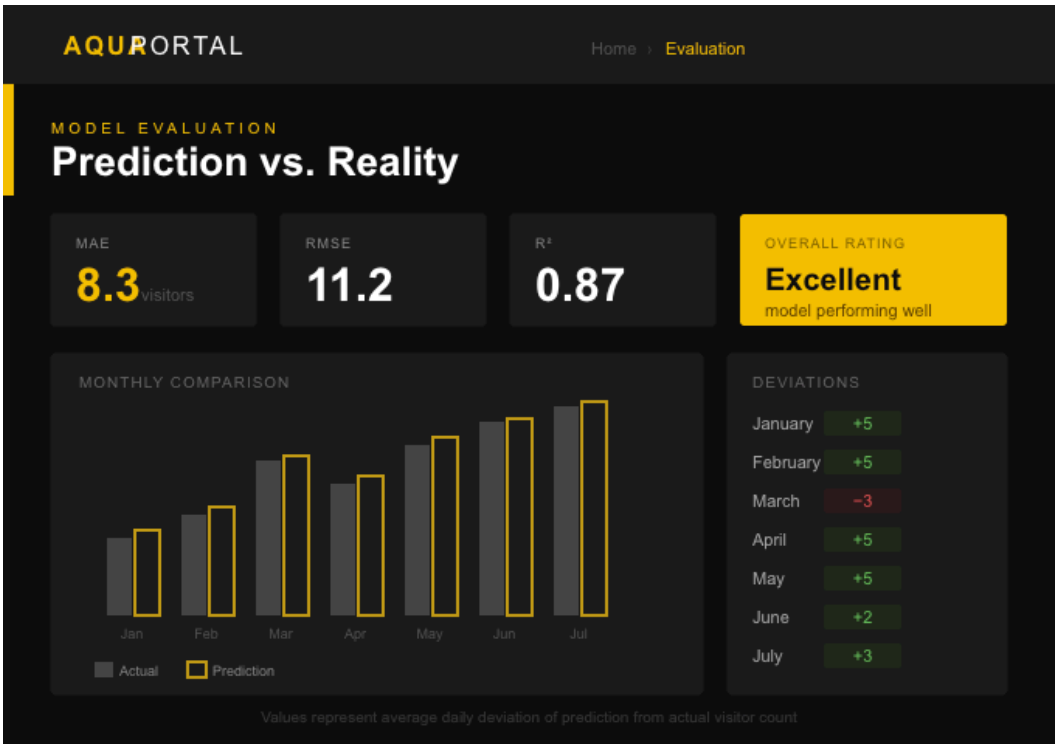

Figure 9 - Course evaluation module design

The graphical design represents a conceptual prototype of the user interface. It serves to illustrate the structure and functionality of the system and does not necessarily correspond to the final implementation.

5. SYSTEM ARCHITECTURE (THREE-TIER ARCHITECTURE)

The proposed system is designed according to the principles of a three-tier architecture, which separates the application into the presentation, application, and data layers. The purpose of this separation is to ensure a clear separation of concerns, reduce coupling between components, and simplify future maintenance and scalability.

The architecture enables seamless integration of AI components—visitor traffic prediction, course recommendation, and anomaly detection—without mixing presentation logic with computational or data processing logic. Visitor traffic prediction is generated periodically (once per day) and subsequently made available to users through the web interface.

5.1. Presentation Layer

5.1.1. Conceptual Design

The presentation layer provides user interaction with the system through a web interface. Its primary responsibilities include:

- displaying visitor traffic predictions (e.g., charts or summaries),
- providing a form for entering user preferences for course recommendation,
- displaying recommended courses,
- enabling access to the administrative interface,
- presenting model evaluation results.

This layer does not contain any prediction or recommendation logic. It acts solely as an interface between the user and the application layer, with all operations executed via API calls.

Additionally, the presentation layer is responsible for:

- basic input validation,
- sending requests to the application layer,
- processing and displaying responses,
- informing users about errors or system states.

5.1.2. Technical Implementation

The presentation layer is implemented as a web interface using HTML, CSS, and JavaScript. Communication with the application layer is handled via REST API using JSON format.

Visitor traffic predictions are visualized dynamically based on data retrieved from the API. The administrative interface is protected by authentication and allows management of course data, system parameters, and model evaluation.

The layer is designed to remain independent of the internal implementation of AI models. Changes in prediction or recommendation logic do not require modifications to the UI, as long as the API contract remains unchanged.

5.2. Application Layer

5.2.1. Conceptual Design

The application layer represents the core of the system. It processes requests from the presentation layer, manages data flow, and integrates AI components.

It consists of the following modules:

- visitor traffic prediction module,
- course recommendation module,
- anomaly detection module,
- system configuration module,
- REST API interface.

Visitor traffic prediction is generated periodically (once per day at midnight). The model processes historical data and produces a 7-day hourly forecast. The results are stored in the database and exposed via API. The recommendation module evaluates course suitability based on user inputs (age, skill level, time availability) and returns a limited set of results (e.g., top 3). The anomaly detection module compares predicted and actual attendance. When a defined threshold is exceeded, it records an alert in the system.

The application layer is responsible for:

- input validation,
- orchestration of internal modules,
- communication with the data layer,
- error handling and logging.

5.2.2. Technical Implementation

The application layer is implemented as a backend service using Python and the FastAPI framework.

The system exposes REST API endpoints for:

- retrieving visitor traffic predictions,
- retrieving recommended courses,
- managing course data,
- configuring system parameters,
- accessing model evaluation results.

Prediction updates are executed by a scheduled task (scheduler), which runs periodically (e.g., once per day).

The application layer includes logging and exception handling mechanisms. In case of prediction generation failure, the system retains the last available prediction (fallback mechanism).

The backend follows the Controller–Service–Repository architecture pattern:

- Controller handles API requests,
- Service contains business logic and AI components,
- Repository manages database access.

The application layer can be deployed as a containerized service using Docker.

5.2.3. AI Components

The application layer includes three main AI components: visitor traffic prediction, course recommendation, and anomaly detection. These components are implemented as independent backend modules.

Visitor Traffic Prediction

The prediction problem is formulated as a time series regression task. Input features include calendar-based attributes (hour, day of week, weekend, month).

A RandomForestRegressor model is used to capture non-linear relationships without requiring feature scaling.

Data are split chronologically (e.g., 80/20) to preserve temporal dependencies. Model validation is performed using TimeSeriesSplit. Performance is evaluated using MAE and RMSE metrics.

The output is a time series of predicted values exposed via REST API.

Course Recommendation

User inputs (age, skill level, time availability) are transformed into numerical features. A RandomForestClassifier model is used to estimate course suitability.

The model returns a ranked list of recommended courses (e.g., top 3). Data are split using train/test split with stratification. Performance is evaluated using accuracy and analyzed via a confusion matrix.

Anomaly Detection

Anomaly detection is implemented using the Isolation Forest algorithm. The model processes derived features such as absolute and relative deviation between predicted and actual attendance.

The model is trained on historical data, assuming most observations represent normal behavior. The contamination parameter defines the expected anomaly ratio. Detected anomalies are stored in the database and made available to administrators.

5.3. Data Layer

5.3.1. Conceptual Design

The data layer ensures persistence and management of all system data. It stores:

- historical attendance data,
- generated predictions,
- course data,
- system configuration parameters,
- anomaly logs and alerts,
- model evaluation metrics.

The layer provides a standardized interface for data access and ensures data consistency.

It does not contain business logic or presentation elements.

5.3.2. Technical Implementation

The data layer is implemented using a relational database (e.g., PostgreSQL).

Database access is handled via an ORM layer, enabling mapping between application entities and database tables.

The database includes tables such as:

- attendance,
- prediction,
- courses,
- system_parameters,
- deviation_log,
- model_metrics.

Regular database backups are supported based on system configuration.

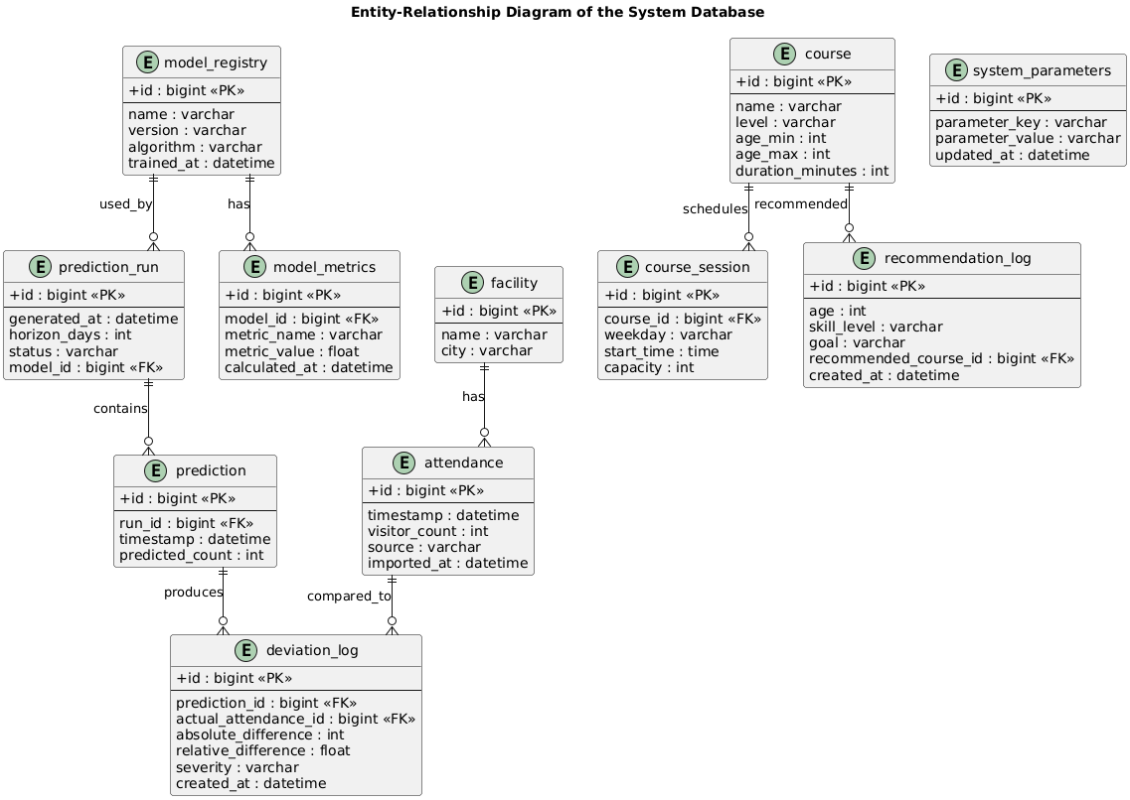

Figure 10- Entity-relationship diagram

5.4. Data Flow

Data flow between system layers is structured as follows:

1. The user sends a request via the presentation layer (e.g., prediction or recommendation request).
2. The presentation layer forwards the request to the application layer via REST API.
3. The application layer processes the request:

- retrieves data from the data layer,
- evaluates the model or uses precomputed results,
- prepares a JSON response.

4. The response is returned to the presentation layer.
5. The presentation layer displays the result to the user.

For periodic prediction updates:

1. The scheduler retrieves historical data from the database.
2. A new prediction is generated.
3. The result is stored in the database.
4. Subsequent user requests use the updated data.

This structure ensures clear separation of responsibilities and controlled data flow across the system.

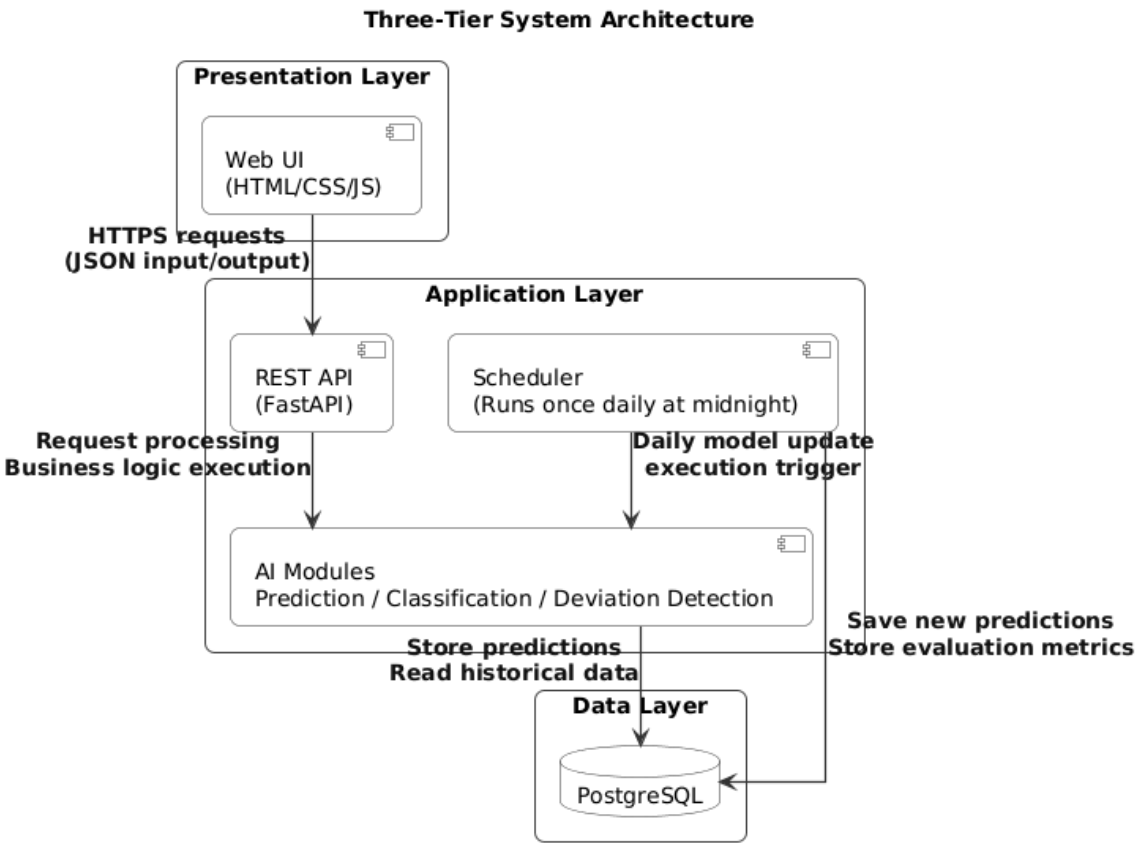

Figure 11– Three-tier system architecture

6. SYSTEM DEPLOYMENT AND OPERATIONS

The system is designed to be easily deployable in a cloud environment, with an emphasis on cost efficiency and sufficient reliability for operating a single swimming facility.

The application layer is containerized using Docker and deployed as an independent service in a cloud environment. For this project, the AWS platform is considered suitable, where the backend can be deployed on services such as Amazon EC2 or Amazon ECS, depending on scalability requirements.

The data layer is implemented using a PostgreSQL database, which can be deployed either as a standalone container or as a managed service (e.g., Amazon RDS). Given the system scope (single facility), a simple and cost-effective deployment strategy is preferred.

Application deployment is automated using a CI/CD pipeline (e.g., GitHub Actions), which handles Docker image build, execution of basic tests, and deployment to the target environment.

To ensure reliable system operation, logging and monitoring mechanisms are implemented. Application logs are collected and stored (e.g., using AWS CloudWatch), enabling retrospective analysis of errors and system behavior. Additionally, system metrics (such as request count, API response time, and prediction performance) can be collected using tools like Prometheus and visualized in Grafana dashboards.

The architecture supports horizontal scalability if needed; however, given the expected workload, the system is intended to operate on a single instance with the possibility of future scaling.

System security is ensured through HTTPS communication and proper handling of sensitive configuration data using environment variables.

7. TEST CASE DESIGN FOR USE CASES

7.1. Testing UC-01 – Display Traffic Predictions

Table 18 – Test Case – Prediction Endpoint Output and Performance Validation

Test ID	TC-UC01-01
Precondition	The database contains the latest prediction for a 7-day horizon, including the last update timestamp. The API is available.
Input	Calling the prediction endpoint (GET /prediction).
Test Steps	1. Call GET /prediction. 2. Measure response time. 3. Verify response structure (list of data points + lastUpdated). 4. Verify number of points and value ranges.
Expected Result	(1) API response time < 3 s. (2) Response contains exactly 168 time points (7×24). (3) Each point contains a timestamp and predicted_count (numeric). (4) All predicted_count values are ≥ 0. (5) UI clearly indicates that the output is an estimate (e.g., "Prediction is an estimate").

Table 19 – Test Case – System Behavior During Prediction Service Unavailability

Test ID	TC-UC01-02
Precondition	Prediction API is unavailable or returns HTTP 5xx error.
Input	GET /prediction during simulated outage.
Test Steps	1. Simulate prediction service failure. 2. Call endpoint. 3. Verify system response.
Expected Result	(1) System returns controlled error or fallback response. (2) UI displays a clear message about unavailability. (3) No application crash and no invalid JSON output.

Table 20 – Test Case – Fallback to Last Available Prediction

Test ID	TC-UC01-03
Precondition	New prediction is not available, but last valid prediction exists in DB.
Input	GET /prediction in fallback mode.
Test Steps	1. Force state without new prediction. 2. Call endpoint. 3. Verify returned data.
Expected Result	(1) Last stored prediction (168 points) is returned. (2) Response includes last update timestamp. (3) Timestamps are hourly and without duplicates.

7.2. Testing UC-02 – Assignment Course Preferences

Table 21 - Test Case – Submission of Valid Course Preferences

Test ID	TC-UC02-01
Precondition	Preferences form and recommendation API are available.
Input	Valid preferences (e.g., age=25, level=beginner, goal=fitness, time=evening).
Test Steps	1. Open form page. 2. Fill required fields. 3. Submit form. 4. Verify API request and JSON response.
Expected Result	(1) Form submits successfully. (2) API returns valid JSON. (3) Response time < 3 s. (4) UI transitions to recommendation display.

Table 22 – Test Case – Validation of Invalid Input Data

Test ID	TC-UC02-02
Precondition	Input validation is implemented.
Input	Invalid data (e.g., missing age, invalid values).
Test Steps	1. Open form.

	2. Enter invalid data. 3. Submit. 4. Verify UI/API behavior.
Expected Result	(1) Form is not submitted (or API returns 400). (2) UI shows validation error message. (3) No inconsistent data is stored.

7.3. Testing UC-03 – Display Recommended Course

Table 23 – Test Case – Structure and Count of Recommended Course

Test ID	TC-UC03-01
Precondition	Courses exist in DB and model is available.
Input	User preferences (JSON).
Test Steps	1. Send valid preferences. 2. Verify response structure. 3. Verify number of recommendations. 4. Verify existence in DB.
Expected Result	(1) API returns 1–3 recommendations. (2) Each item contains: courseId, name, level (optional score). (3) No duplicates. (4) All courseId exist in DB.

Table 24 – Test Case – Behavior When No Suitable Course Are Available

Test ID	TC-UC03-02
Precondition	No suitable courses exist.
Input	Edge-case preferences.
Test Steps	1. Send extreme preferences. 2. Verify response and UI.
Expected Result	(1) API returns 0–2 results. (2) UI displays message (e.g., “No suitable courses found”). (3) Response is always valid JSON.

7.4. Testing UC-04 – Manage Course

Table 25 – Test Case – Create or Update Course

Test ID	TC-UC04-01
Precondition	Admin access and DB available.
Input	New or edited course data.
Test Steps	1. Open admin section. 2. Create/update course. 3. Save changes. 4. Reload list.
Expected Result	(1) Course saved successfully. (2) Visible after reload. (3) Data consistent.

Table 26 – Test Case – Validation of Invalid Course Data

Test ID	TC-UC04-02
Precondition	Validation implemented.
Input	Invalid course data.
Test Steps	1. Enter invalid data. 2. Attempt save.
Expected Result	(1) System rejects input. (2) Validation message displayed. (3) No invalid DB record created.

7.5. Testing UC-05 – Configure System Parameters

Table 27 – Test Case – Update Systems Configuration Parameter

Test ID	TC-UC05-01
Precondition	Parameter exists in DB.
Input	Parameter update (e.g., contamination=0.03).
Test Steps	1. Open parameters section. 2. Modify value. 3. Save.

	4. Verify DB. 5. Trigger dependent process.
Expected Result	(1) Value stored with updated timestamp. (2) New value used in processing.

Table 28 – Test Case – Validation of Invalid Parameter Value

Test ID	TC-UC05-02
Precondition	Validation implemented.
Input	Invalid parameter value.
Test Steps	1. Enter invalid value. 2. Save.
Expected Result	(1) Change rejected. (2) Error message displayed. (3) Original value remains in DB.

7.6. Testing UC-06 – Update Prediction

Table 29 – Test Case – Successful Scheduled Prediction Update

Test ID	TC-UC06-01
Precondition	Scheduler configured, DB available.
Input	Scheduled job execution.
Test Steps	1. Trigger job. 2. Verify processing. 3. Check DB.
Expected Result	(1) New prediction stored. (2) 168 points generated. (3) Valid timestamps. (4) Values ≥ 0. (5) Job status = success.

Table 30 – Test Case – Behavior on Prediction Update Failure

Test ID	TC-UC06-02
Precondition	Previous prediction exists.

Input	Job execution with failure.
Test Steps	1. Simulate failure. 2. Run job. 3. Check logs and DB.
Expected Result	(1) Job fails and logs error. (2) No corrupted prediction created. (3) Last valid prediction remains available.

7.7. Testing UC-07 – Display Model Evaluation

Table 31 – Test Case – Display of Model Evaluation Metrics

Test ID	TC-UC07-01
Precondition	Metrics exist in DB.
Input	GET /model/metrics
Test Steps	1. Open admin metrics page. 2. Load data. 3. Verify values.
Expected Result	(1) MAE and RMSE displayed. (2) Values numeric and ≥ 0. (3) Model version and timestamp shown. (4) No inconsistent data.

Table 32 – Test Case – Behavior When Metrics Are Unavailable

Test ID	TC-UC07-02
Precondition	No metrics stored.
Input	GET /model/metrics
Test Steps	1. Remove metrics. 2. Load page.
Expected Result	(1) UI shows “Metrics not available”. (2) API returns controlled response. (3) System remains functional.

CONCLUSION

The objective of this documentation was to systematically design an intelligent extension of a swimming facility web portal by integrating AI components, and to define its architecture, requirements, and validation approach.

Based on the conducted analyses and the proposed system design, it can be concluded that the solution is:

- technically feasible,
- architecturally consistent,
- testable through measurable criteria,
- scalable and extendable for future development.

The proposed three-tier architecture ensures a clear separation between presentation, application, and data layers, which simplifies system maintenance, improves modularity, and supports future enhancements. The AI components are implemented as independent backend modules, and their performance is evaluated using standard metrics (MAE, RMSE, accuracy) and validated on real operational data.

The system is designed with production deployment in mind, including containerization using Docker, automated CI/CD pipelines, and deployment in a cloud environment (e.g., AWS). Monitoring and logging mechanisms (e.g., CloudWatch, Prometheus, Grafana) provide visibility into system behavior and support operational reliability.

The defined test cases cover:

- correctness of outputs (e.g., exactly 168 hours of prediction),
- performance requirements (API response time < 3 seconds),
- handling of error states and system failures,
- data integrity and validation,
- fallback mechanisms in case of prediction update failure.

This document represents a complete design foundation for system implementation and demonstrates the practical application of software engineering and machine learning principles in a real-world domain context.